

9. Проектирование баз данных и работа с ними Веб-приложений. Введение в БД, SQL Server, ADO.NET

В данной лекции рассматриваются вопросы проектирования баз данных и работы с ними Веб-приложений. Дается введение в технологию доступа к данным ADO.NET

Содержание

- 9.1. Проектирование баз данных
 - 9.1.1. Понятие базы данных
 - 9.1.2. Классификация БД
 - 9.1.2.1. Классификация БД по модели данных
 - 9.1.2.1.1. Иерархические
 - 9.1.2.1.2. Сетевые
 - 9.1.2.1.3. Реляционные
 - 9.1.2.1.4. Постреляционные
 - 9.1.2.1.5. Объектно-ориентированные
 - 9.1.2.1.6. Объектно-реляционные
 - 9.1.3. Основные понятия реляционных БД
 - 9.1.3.1. Таблицы
 - 9.1.3.2. Первичные и внешние ключи
 - 9.1.3.3. Индексирование
 - 9.1.3.4. Нормальные формы
 - 9.1.3.4.1. Первая нормальная форма (1NF)
 - 9.1.3.4.2. Вторая нормальная форма (2NF)
 - 9.1.3.4.3. Третья нормальная форма (3NF)
 - 9.1.3.4.4. Нормальная форма Бойса-Кодда (BCNF)
 - 9.1.3.6. Транзакции
 - 9.1.4. Проектирование баз данных
 - 9.1.5. Технологии для доступа к базам данных в Веб
 - 9.1.5.1. Преобразование содержимого БД в статические документы
 - 9.1.5.2. Динамическое создание гипертекстовых документов
 - 9.1.5.3. Создание информационного хранилища на основе СУБД
 - 9.1.6. Язык SQL
 - 9.1.7. Ключевые термины
- 9.2. Доступ к данным в .NET
 - 9.2.1. СУБД MS SQL Server 2008
 - 9.2.1.1. SQL Server 2005
 - 9.2.1.2. Новинки SQL Server 2008
 - 9.2.2. ADO.NET
 - 9.2.2.1. Общие сведения
 - 9.2.2.2. Организация взаимодействия с БД
 - 9.2.2.2.1. Подключение к БД
 - 9.2.2.2.2. Выполнение команд над наборами данных
 - 9.2.2.2.3. Использование параметризованных команд
 - 9.2.2.2.4. Использование хранимых процедур
 - 9.2.2.3. Отсоединенные наборы данных
 - 9.2.2.3.1. Класс DataSet
 - 9.2.2.3.2. Класс DataAdapter
 - 9.2.2.4. Привязка и отображение данных
 - 9.2.2.4.1. Привязка с одним значением
 - 9.2.2.4.2. Привязка с множественным значением
 - 9.2.2.5. Использование DataView для фильтрации и сортировки данных
 - 9.2.2.5.1. Сортировка данных
 - 9.2.2.5.2. Фильтрация данных

9.2.3. Ключевые термины

9.3. Краткие итоги

9.1. Проектирование баз данных

9.1.1. Понятие базы данных

Большинство автоматизированных систем тем или иным образом используют структурированную информацию. В современных приложениях такие упорядоченные данные принято хранить в базах данных – особых файлах, использование которых вместе со специальными программными средствами позволяет пользователю, как просматривать необходимую информацию, так и манипулировать ею.

Например, телефонный справочник представляет собой простейшую базу данных, в которой вся информация располагается в единственной таблице [1, 2]. Каждая запись в этой таблице содержит идентификатор конкретного человека – ФИО и его номер телефона. Таким образом, таблица состоит из записей, информация в которых разделена на несколько частей – полей (табл. 9.1).

Таблица 9.1. Телефонный справочник

ФИО	Номер телефона
Иванов Иван Иванович	532-43-12
Ильин Федор Иванович	321-32-34

Рассмотрим различные определения базы данных:

- Базой данных является представленная в объективной форме совокупность самостоятельных материалов, систематизированных таким образом, чтобы эти материалы могли быть найдены и обработаны с помощью электронной вычислительной машины [3].
- База данных – организованная в соответствии с определенными правилами и поддерживаемая в памяти компьютера совокупность данных, характеризующая актуальное состояние некоторой предметной области и используемая для удовлетворения информационных потребностей пользователей [4].
- База данных – совокупность данных, хранимых в соответствии со схемой данных, манипулирование которыми выполняют в соответствии с правилами средств моделирования данных [5].
- База данных – некоторый набор перманентных (постоянно хранимых) данных, используемых прикладными программными системами какого-либо предприятия [6].
- База данных – совокупность взаимосвязанных данных, совместно хранимых в одном или нескольких компьютерных файлах [7].
- База данных – совместно используемый набор логически связанных данных (и описание этих данных), предназначенный для удовлетворения информационных потребностей организации [8].
- База данных – это совокупность связанных данных, организованных по определенным правилам, предусматривающим общие принципы описания, хранения и манипулирования, независимая от прикладных программ [9].

Существует множество и других определений, отражающих скорее субъективное мнение тех или иных авторов о том, что означает этот термин в их понимании, однако общепризнанная единая формулировка отсутствует. Наиболее часто используются следующие отличительные признаки [10]:

База данных хранится и обрабатывается в вычислительной системе. Таким образом, любые некомпьютерные хранилища информации (архивы, библиотеки, картотеки и т. п.) базами данных не являются.

Данные в базе данных логически структурированы (систематизированы) с целью обеспечения возможности их эффективного поиска и обработки в вычислительной системе. Структурированность подразумевает явное выделение составных частей (элементов), связей между ними, а также типизацию элементов и связей, при которой с типом элемента (связи) соотносится определенная семантика и допустимые операции.

База данных включает метаданные, описывающие логическую структуру БД в формальном виде (в соответствии с некоторой метамodelью). В соответствии с ГОСТ Р ИСО МЭК ТО 10032-2007, "постоянные данные в среде базы данных включают в себя схему и базу данных. Схема включает в себя описания содержания, структуры и ограничений целостности, используемые для создания и поддержки базы данных. База данных включает в себя набор постоянных данных, определенных с помощью схемы. Система управления данными использует определения данных в схеме для обеспечения доступа и управления доступом к данным в базе данных" [5].

Из перечисленных признаков только первый является строгим, а другие допускает различные трактовки и различные степени оценки. Можно лишь установить некоторую степень соответствия требованиям к БД.

Многие специалисты указывают на распространенную ошибку, состоящую в некорректном использовании термина база данных вместо термина система управления базами данных. Эти понятия, следовательно, необходимо различать.

Система управления базами данных (СУБД) – это специализированная программа (чаще комплекс программ), предназначенная для организации и ведения базы данных [11].

Таким образом, система управления базой данных (СУБД) – важнейший компонент информационной системы. Для создания и управления информационной системой СУБД необходима в той же степени, как для разработки программы на алгоритмическом языке необходим транслятор.

Основные функции СУБД [12]:

- управление данными во внешней памяти (на дисках);
- управление данными в оперативной памяти;
- журналирование изменений и восстановление базы данных после сбоев;
- поддержание языков БД (язык определения данных, язык манипулирования данными).

Обычно современная СУБД содержит следующие компоненты [12] (рис. 9.1):

- ядро, которое отвечает за управление данными во внешней и оперативной памяти и журналирование;
- процессор языка базы данных, обеспечивающий оптимизацию запросов на извлечение и изменение данных и создание, как правило, машинно-независимого исполняемого внутреннего кода;
- подсистему поддержки времени исполнения, которая интерпретирует программы манипуляции данными, создающие пользовательский интерфейс с СУБД;
- сервисные программы (внешние утилиты), обеспечивающие ряд дополнительных возможностей по обслуживанию информационной системы.

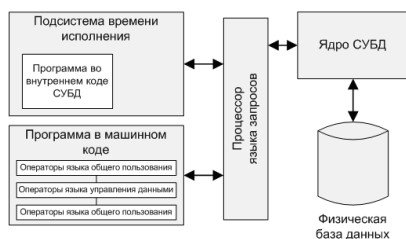


Рис. 9.1. Компоненты СУБД

На рис. 9.2 приведены основные функции и содержимое компонентов обработки баз данных [13].



Рис. 9.2. Функции компонентов обработки баз данных

9.1.2. Классификация БД

Существует огромное количество разновидностей баз данных, отличающихся по различным критериям (например, в "Энциклопедии технологий баз данных" М. Р. Когаловского [4] определяются свыше 50 видов БД).

Базы данных могут быть классифицированы [10]:

- По технологии хранения:
- БД во вторичной памяти (традиционные);
- БД в оперативной памяти (in-memory databases);
- БД в третичной памяти (tertiary databases);
- По содержимому:
- Географические;
- Исторические;
- Научные;
- Мультимедийные;
- По степени распределенности:
- Централизованные (сосредоточенные);
- Распределенные.

Подробнее же рассмотрим классификацию БД по моделям данных.

9.1.2.1. Классификация БД по модели данных

В классической теории баз данных, модель данных есть формальная теория представления и обработки данных в системе управления базами данных (СУБД), которая включает, по меньшей мере, три аспекта [12, 14]:

- структура данных – описывает точку зрения пользователя на представление данных;
- набор допустимых операций, выполняемых на структуре данных (модель данных предполагает, как минимум, наличие языка определения данных (ЯОД), описывающего структуру их хранения, и языка манипулирования данными (ЯМД), включающего операции извлечения и модификации данных);
- ограничения целостности – механизм поддержания соответствия данных предметной области на основе формально описанных правил.

Другими словами, структура данных определяет, что из себя логически представляет база данных, ограничения целостности определяют средства описаний корректных состояний базы данных, набор допустимых операций определяет способы перехода между состояниями базы данных (то есть способы модификации данных) и способы извлечения данных из базы данных [14].

Каждая БД и СУБД строится на основе некоторой явной или неявной модели данных. Все СУБД, построенные на одной и той же модели данных, относят к одному типу. Например, основой реляционных СУБД является реляционная модель данных, сетевых СУБД – сетевая модель данных и т.д.

Длительное время термин "модель данных" использовался без формального определения. Одним из первых специалистов, который достаточно формально определил это понятие, был Э. Кодд. В статье "Модели данных в управлении базами данных" [15] он определил модель данных как комбинацию трех компонентов:

- коллекции типов объектов данных, образующих базовые строительные блоки для любой базы данных, соответствующей модели;
- коллекции общих правил целостности, ограничивающих набор экземпляров тех типов объектов, которые законным образом могут появиться в любой такой базе данных;
- коллекции операций, применимых к таким экземплярам объектов для выборки и других целей.

В процессе исторического развития в СУБД использовались следующие модели данных [12]:

9.1.2.1.1. Иерархические

Иерархическая модель базы данных состоит из объектов с указателями от родительских объектов к потомкам, соединяя вместе связанную информацию.

Организация данных в СУБД иерархического типа определяется в следующих терминах [12]:

Атрибут (элемент данных) – наименьшая единица структуры данных. Обычно каждому элементу при описании базы данных присваивается уникальное имя. По этому имени к нему обращаются при обработке. Элемент данных также часто называют полем.

Запись – именованная совокупность атрибутов. Использование записей позволяет за одно обращение к базе получить некоторую логически связанную совокупность данных. Именно записи изменяются, добавляются и удаляются. Тип записи определяется составом ее атрибутов. Экземпляр записи – конкретная запись с конкретным значением элементов

Групповое отношение – иерархическое отношение между записями двух типов. Родительская запись (владелец группового отношения) называется исходной записью, а дочерние записи (члены группового отношения) – подчиненными. Иерархическая база данных может хранить только такие древовидные структуры.

Одной из наиболее популярных иерархических СУБД была Information Management System (IMS) компании IBM, появившаяся в 1968 году [13].

9.1.2.1.2. Сетевые

Если структура данных оказывалась сложнее, чем обычная иерархия, простота структуры иерархической базы данных становилась ее недостатком [16].

Основные принципы сетевой модели данных были разработаны в середине 60-х годов, эталонный вариант сетевой модели данных описан в отчетах рабочей группы по языкам баз данных (COConference on DATA SYstem Languages) CODASYL в 1971 г. [12].

Сетевая модель данных определяется в тех же терминах, что и иерархическая.

Основное различие этих моделей состоит в том, что в сетевой модели запись может быть членом более чем одного группового отношения. Согласно этой модели каждое групповое отношение именуется и проводится различие между его типом и экземпляром. Тип группового отношения задается его именем и определяет свойства общие для всех экземпляров данного типа. Экземпляр группового отношения представляется записью-владельцем и множеством (возможно пустым) подчиненных записей. При этом имеется следующее ограничение: экземпляр записи не может быть членом двух экземпляров групповых отношений одного типа.

Основным недостатком сетевых баз данных (как и иерархических) является то, что они очень "жесткие" [16]. Изменение структуры базы данных обычно означало перестройку всей базы данных.

9.1.2.1.3. Реляционные

Недостатки иерархической и сетевой моделей привели к появлению новой, реляционной модели данных, созданной Э. Коддом в 1970 году [16]. Кодд первым предложил использовать для обработки данных аппарат теории множеств (объединение, пересечение, разность, декартово произведение) и показал, что любое представление данных сводится к совокупности двумерных таблиц особого вида, известного в математике как отношение [1].

Реляционная модель данных (РМД) – логическая модель данных, прикладная теория, описывающая структурный аспект, аспект целостности и аспект обработки данных в реляционных базах данных [17]:

- структурный аспект – данные в базе данных представляют собой набор отношений;
- аспект целостности – отношения (таблицы) отвечают определенным условиям целостности (РМД поддерживает декларативные ограничения целостности уровня домена (типа данных), уровня отношения и уровня базы данных);
- аспект обработки – РМД поддерживает операторы манипулирования отношениями (реляционная алгебра, реляционное исчисление).

Кроме того, в состав реляционной модели данных обычно включают теорию нормализации.

Для лучшего понимания РМД следует отметить три важных обстоятельства:

- модель является логической, то есть отношения являются логическими (абстрактными), а не физическими (хранимыми) структурами;
- для реляционных баз данных верен информационный принцип: все информационное наполнение базы данных представлено одним и только одним способом, а именно – явным заданием значений атрибутов в кортежах отношений; в частности, нет никаких указателей (адресов), связывающих одно значение с другим;
- наличие реляционной алгебры позволяет реализовать декларативное программирование и декларативное описание ограничений целостности, в дополнение к навигационному (процедурному) программированию и процедурной проверке условий.

К сожалению, практическое определение понятия "реляционная база данных" оказалось гораздо более расплывчатым, чем точное математическое определение, данное этому термину Коддом в 1970 году. В первых реляционных СУБД не были реализованы некоторые из ключевых частей модели Кодда, и этот пробел был восполнен только впоследствии. По мере роста популярности реляционной концепции реляционными стали называться многие базы данных, которые на деле таковыми не являлись.

В ответ на неправильное использование термина "реляционный" Кодд в 1985 году написал статью, где сформулировал 12 правил, которым должна удовлетворять любая база данных, претендующая на звание реляционной [15]:

1. Правило информации. Вся информация в базе данных должна быть предоставлена исключительно на логическом уровне и только одним способом – в виде значений, содержащихся в таблицах.

2. Правило гарантированного доступа. Логический доступ ко всем и каждому элементу данных (атомарному значению) в реляционной базе данных должен обеспечиваться путем использования комбинации имени таблицы, первичного ключа и имени столбца.

3. Правило поддержки недействительных значений. В реляционной базе данных должна быть реализована поддержка недействительных значений, которые отличаются от строки символов нулевой длины, строки пробельных символов и от нуля или любого другого числа и используются для представления отсутствующих данных независимо от типа этих данных.

4. Правило динамического каталога, основанного на реляционной модели. Описание базы данных на логическом уровне должно быть представлено в том же виде, что и основные данные, чтобы пользователи, обладающие соответствующими правами, могли работать с ним с помощью того же реляционного языка, который они применяют для работы с основными данными.

5. Правило исчерпывающего подязыка данных. Реляционная система может поддерживать различные языки и режимы взаимодействия с пользователем, однако должен существовать, по крайней мере, один язык, операторы которого можно представить в виде строк символов в соответствии с некоторым четко определенным синтаксисом и который в полной мере поддерживает следующие элементы:

- определение данных;
- определение представлений;
- обработку данных (интерактивную и программную);
- условия целостности;
- идентификация прав доступа;
- границы транзакций (начало, завершение и отмена).

6. Правило обновления представлений. Все представления, которые теоретически можно обновить, должны быть доступны для обновления.

7. Правило добавления, обновления и удаления. Возможность работать с отношением как с одним операндом должна существовать не только при чтении данных, но и при добавлении, обновлении и удалении данных.

8. Правило независимости физических данных. Прикладные программы и утилиты для работы с данными должны на логическом уровне оставаться нетронутыми при любых изменениях способов хранения данных или методов доступа к ним.

9. Правило независимости логических данных. Прикладные программы и утилиты для работы с данными должны на логическом уровне оставаться

нетронутыми при внесении в базовые таблицы любых изменений, которые теоретически позволяют сохранить нетронутыми содержащиеся в этих таблицах данные.

10. Правило независимости условий целостности. Должна существовать возможность определять условия целостности, специфические для конкретной реляционной базы данных, на подязыке реляционной базы данных и хранить их в каталоге, а не в прикладной программе.
11. Правило независимости распространения. Реляционная СУБД не должна зависеть от потребностей конкретного клиента.
12. Правило единственности. Если в реляционной системе есть низкоуровневый язык (обрабатывающий одну запись за один раз), то должна отсутствовать возможность использования его для того, чтобы обойти правила и условия целостности, выраженные на реляционном языке высокого уровня (обрабатывающем несколько записей за один раз).

Однако на практике можно пользоваться и более простым определением: реляционной называется база данных, в которой все данные, доступные пользователю, организованы в виде таблиц, а все операции над данными сводятся к операциям над этими таблицами.

Приведенное определение не оставляет места встроенным указателям, имеющимся в иерархических и сетевых СУБД.

9.1.2.1.4. Постреляционные

Постреляционная модель данных представляет собой расширенную реляционную модель, в которой отменено требование атомарности атрибутов [12]. Поэтому постреляционную модель называют "не первой нормальной формой" (NF2) или "многомерной базой данных". Она использует трехмерные структуры, позволяя хранить в полях таблицы другие таблицы. Тем самым расширяются возможности по описанию сложных объектов реального мира. В качестве языка запросов используется несколько расширенный SQL, позволяющий извлекать сложные объекты из одной таблицы без операций соединения.

Существует несколько коммерческих постреляционных СУБД, самыми известными из которых являются системы Adabas, Pick и Universe.

9.1.2.1.5. Объектно-ориентированные

В основу объектной модели положена концепция объектно-ориентированного программирования, в которой данные представляются в виде набора объектов и классов, связанных между собой родственными отношениями, а работа с объектами осуществляется с помощью скрытых (инкапсулированных) в них методов [1].

Объектно-ориентированная база данных (ООБД) – база данных, в которой данные оформлены в виде моделей объектов, включающих прикладные программы, которые управляются внешними событиями [18]. Результатом совмещения возможностей (особенностей) баз данных и возможностей объектно-ориентированных языков программирования являются Объектно-ориентированные системы управления базами данных (ООСУБД). ООСУБД позволяет работать с объектами баз данных так же, как с объектами в программировании в объектно-ориентированных языках программирования.

Некоторые объектно-ориентированные базы данных разработаны для плотного взаимодействия с такими объектно-ориентированными языками программирования как Python, Java, C#, Visual Basic .NET, C++, Objective-C и Smalltalk; другие имеют свои собственные языки программирования (например, Cache Object Script).

Объектно-ориентированные базы данных обычно рекомендованы для тех случаев, когда требуется высокопроизводительная обработка данных, имеющих сложную структуру.

Примеры объектно-ориентированных СУБД: Cache, GemStone, ONTOS.

9.1.2.1.6. Объектно-реляционные

В последнее время производители СУБД стремятся соединить два этих подхода и проповедуют объектно-реляционную модель представления данных [1]. Примеры таких СУБД – IBM DB2 for Common Servers, Oracle 10, SQL Server 2008.

Один из способов объединения возможностей реляционного и объектно-ориентированного подхода к управлению данными предложил известный американский ученый Майкл Стоунбрейкер [12]. Согласно его воззрениям реляционную СУБД нужно просто дополнить средствами доступа к сложным данным. При этом ядро СУБД не требует переработки, как в случае с SQL3, и сохраняет все присущие реляционным системам достоинства. Объектные расширения реализуются в виде надстроек, которые динамически подключаются к ядру.

9.1.3. Основные понятия реляционных БД

9.1.3.1. Таблицы

В реляционной базе данных информация организована в виде таблиц, разделенных на строки и столбцы, на пересечении которых содержатся значения данных [16]. У каждой таблицы имеется уникальное имя, описывающее ее содержимое.

Столбы таблицы имеют следующие свойства:

- все значения, содержащиеся в одном и том же столбце, являются данными одного типа;
- множество значений, которые могут содержаться в столбце, называется доменом этого столбца;
- у каждого столбца в таблице есть свое, уникальное в пределах одной таблицы, имя, которое обычно служит заголовком столбца;
- столбцы таблицы упорядочены слева направо, и их порядок определяется при создании таблицы;
- в любой таблице всегда есть как минимум один столбец;
- в стандарте ANSI/ISO не указывается максимально допустимое число столбцов в таблице, однако почти во всех коммерческих СУБД этот предел существует.

Строки таблицы имеют следующие свойства:

строки таблицы не имеют определенного порядка;

в таблице может содержаться любое количество строк (в том числе и ни одной);

стандарт ANSI/ISO не накладывает ограничений на количество строк в таблице, и во многих СУБД размер таблиц ограничен лишь свободным дисковым пространством компьютера или весьма высок.

9.1.3.2. Первичные и внешние ключи

Первичный ключ (primary key, PK) – минимальный набор полей, уникально идентифицирующий запись в таблице [1]. В правильно построенной реляционной базе данных в каждой таблице есть один или несколько столбцов, значения в которых во всех строках разные [16] – этот столбец (столбцы) и называется первичным ключом таблицы.

Первичный ключ может представлять собой комбинацию столбцов. Такой первичный ключ называется составным.

Таблица, в которой все строки отличаются друг от друга, в математических терминах называется отношением.

Столбец одной таблицы, значения в котором совпадают со значениями столбца, являющегося первичным ключом другой таблицы, называется внешним ключом.

Внешний ключ, как и первичный ключ, тоже может представлять собой комбинацию столбцов. На практике внешний ключ всегда будет составным (состоящим из нескольких столбцов), если он ссылается на составной первичный ключ в другой таблице. Очевидно, что количество столбцов и их типы данных в первичном и внешнем ключах совпадают.

Если таблица связана с несколькими другими таблицами, она может иметь несколько внешних ключей.

Внешние ключи являются неотъемлемой частью реляционной модели, поскольку реализуют отношения между таблицами базы данных.

На рис. 9.3 показана связь внешнего и первичного ключа.

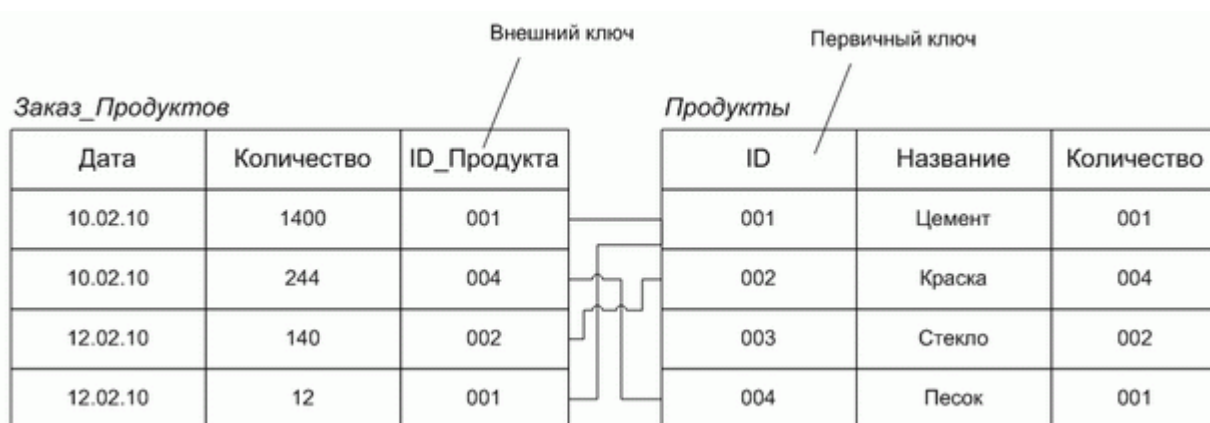


Рис. 9.3. Внешний и первичный ключи

9.1.3.3. Индексирование

Одна из основных задач, возникающих при работе с базами данных, – это задача поиска [1]. При этом, поскольку информации в базе данных, как правило, содержится много, перед программистами встает задача не просто поиска, а эффективного поиска, т.е. поиска за сравнительно небольшое время и с достаточной точностью. Для этого (для оптимизации производительности запросов) производят индексирование некоторых полей таблицы. Использовать индексы полезно для быстрого поиска строк с указанным значением одного столбца. Без индекса чтение таблицы осуществляется по всей таблице, начиная с первой записи, пока не будут найдены соответствующие строки. Если же таблица содержит индекс по рассматриваемым столбцам, то база данных может

быстро определить позицию для поиска в середине файла данных без просмотра всех данных. Это происходит потому, что база данных помещает проиндексированные поля ближе в памяти, так, чтобы можно было быстрее найти их значения. Для таблицы, содержащей 1000 строк, это будет как минимум в 100 раз быстрее по сравнению с последовательным перебором всех записей.

9.1.3.4. Нормальные формы

Нормальная форма – свойство отношения в реляционной модели данных, характеризующее его с точки зрения избыточности, которая потенциально может привести к логически ошибочным результатам выборки или изменения данных [19]. Нормальная форма определяется как совокупность требований, которым должно удовлетворять отношение.

Процесс преобразования базы данных к виду, отвечающему нормальным формам, называется нормализацией. Нормализация предназначена для приведения структуры базы данных к виду, обеспечивающему минимальную избыточность, то есть нормализация не имеет целью уменьшение или увеличение производительности работы или же уменьшение или увеличение объема БД. Конечной целью нормализации является уменьшение потенциальной противоречивости хранимой в БД информации.

Устранение избыточности производится, как правило, за счет декомпозиции отношений таким образом, чтобы в каждом отношении хранились только первичные факты (то есть факты, не выводимые из других хранимых фактов).

Приведем определения нескольких нормальных форм.

9.1.3.4.1. Первая нормальная форма (1NF)

Для определения первой нормальной формы необходимо дать два определения [12]:

Простой атрибут – атрибут, значения которого атомарны (неделимы).

Сложный атрибут – получается соединением нескольких атомарных атрибутов, которые могут быть определены на одном или разных доменах.

Теперь можно дать определение первой нормальной формы:

Отношение находится в 1NF, если значения всех его атрибутов атомарны.

9.1.3.4.2. Вторая нормальная форма (2NF)

Очень часто первичный ключ отношения включает несколько атрибутов. При этом вводится понятие полной функциональной зависимости: неключевой атрибут функционально полно зависит от составного ключа если он функционально зависит от всего ключа в целом, но не находится в функциональной зависимости от какого-либо из входящих в него атрибутов.

Теперь можно дать определение второй нормальной формы [12]:

Отношение находится во 2NF, если оно находится в 1NF, и каждый неключевой атрибут функционально полно зависит от ключа.

9.1.3.4.3. Третья нормальная форма (3NF)

Для определения третьей нормальной формы необходимо ввести понятие транзитивной функциональной зависимости:

Пусть X, Y, Z – три атрибута некоторого отношения. При этом X → Y и Y → Z, но обратное соответствие отсутствует, т.е. Z ↮ Y и Y ↮ X. Тогда Z транзитивно зависит от X.

Теперь можно дать определение третьей нормальной формы [12]:

Отношение находится в 3 NF, если оно находится во 2 NF, и каждый неключевой атрибут нетранзитивно зависит от первичного ключа.

9.1.3.4.4. Нормальная форма Бойса-Кодда (BCNF)

Эта нормальная форма вводит дополнительное ограничение по сравнению с 3 NF [12]:

Отношение находится в BCNF, если оно находится во 3 NF, и в ней отсутствуют зависимости атрибутов первичного ключа от неключевых атрибутов.

9.1.3.6. Транзакции

Понятие транзакции не входит в реляционную модель данных, т.к. транзакции рассматриваются не только в реляционных СУБД, но и в СУБД других типов, а также и в других типах информационных систем. Однако мы рассмотрим его именно в этом разделе, т.к. транзакции очень часто используются в реляционных базах данных.

Транзакция – это неделимая, с точки зрения воздействия на СУБД, последовательность операций манипулирования данными [20]. Для пользователя транзакция выполняется по принципу "все или ничего", т.е. либо транзакция выполняется целиком и переводит базу данных из одного целостного состояния в другое целостное состояние, либо, если по каким-либо причинам, одно из действий транзакции невыполнимо или произошло какое-либо нарушение работы системы, база данных возвращается в исходное состояние, которое было до начала транзакции (происходит откат транзакции).

Транзакция обладает четырьмя важными свойствами, известными как свойства АСИД (ACID) [20, 21]:

- Атомарность (Atomicity). Транзакция выполняется как атомарная операция – либо выполняется вся транзакция целиком, либо она целиком не выполняется.
- Согласованность (Consistency). Транзакция переводит базу данных из одного согласованного (целостного) состояния в другое согласованное (целостное) состояние.
- Изоляция (Isolation). Транзакции разных пользователей не должны мешать друг другу (например, как если бы они выполнялись строго по очереди).
- Долговечность (Durability). Если транзакция выполнена, то результаты ее работы должны сохраниться в базе данных, даже если в следующий момент произойдет сбой системы.

Транзакция обычно начинается автоматически с момента присоединения пользователя к СУБД и продолжается до тех пор, пока не произойдет одно из следующих событий:

- подана команда COMMIT WORK (зафиксировать транзакцию);
- подана команда ROLLBACK WORK (откатить транзакцию);
- произошло отсоединение пользователя от СУБД;
- произошел сбой системы.

Свойства АСИД транзакций не всегда выполняются в полном объеме. Особенно это относится к свойству Изоляция. Различается несколько уровней изоляции транзакций. На низшем уровне изоляции транзакции могут реально мешать друг другу, на высшем они полностью изолированы. За большую изоляцию транзакций приходится платить большими накладными расходами системы и замедлением работы.

Свойство Долговечность также не является абсолютным свойством, т.к. некоторые системы допускают вложенные транзакции. Если транзакция Б запущена внутри транзакции А, и для транзакции Б подана команда COMMIT WORK, то фиксация данных транзакции Б является условной, т.к. внешняя транзакция А может откатиться. Результаты работы внутренней транзакции Б будут окончательно зафиксированы, только если будет зафиксирована внешняя транзакция А.

9.1.4. Проектирование баз данных

Проектирование баз данных – процесс решения класса задач, связанных с созданием баз данных [22].

При выполнении данного процесса решаются следующие основные задачи [22, 23]:

- обеспечение хранения в БД всей необходимой информации;
- обеспечение возможности получения данных по всем необходимым запросам;
- сокращение избыточности и дублирования данных;
- обеспечение целостности данных (правильности их содержания): исключение противоречий в содержании данных, исключение их потери и т.д.

Опишем основные этапы проектирования баз данных [22, 23, 24]:

Концептуальное (инфологическое) проектирование – построение формализованной модели предметной области. Такая модель строится с использованием стандартных языковых средств, обычно графических, например, ER-диаграмм и не ориентируется на какую-либо конкретную СУБД. Основные элементы данной модели:

- описание объектов предметной области и связей между ними;
- описание информационных потребностей пользователей (описание основных запросов к БД);
- описание документооборота. Описание документов, используемых как исходные данные для БД и документов, составляемых на основе БД;
- описание алгоритмических зависимостей между данными;
- описание ограничений целостности, т.е. требований к допустимым значениям данных и к связям между ними.

Логическое (дatalogическое) проектирование – отображение инфологической модели на модель данных, используемую в конкретной СУБД, например на реляционную модель данных. Для реляционных СУБД даталогическая модель – набор таблиц, обычно с указанием ключевых полей, связей между таблицами. Если инфологическая модель построена в виде ER-диаграмм (или других формализованных средств), то даталогическое проектирование представляет собой построение таблиц по определенным формализованным правилам, а также нормализацию этих таблиц.

Физическое проектирование – реализация даталогической модели средствами конкретной СУБД, а также выбор решений, связанных с физической средой хранения данных: выбор методов управления дисковой памятью, методов доступа к данным, методов сжатия данных и т.д. – эти задачи решаются в основном средствами СУБД и скрыты от разработчика БД.

9.1.5. Технологии для доступа к базам данных в Веб

При обеспечении Веб-доступа к существующим БД, возможен ряд путей – комплексов технологических и организационных решений [25]. Практика использования Веб-технологии для доступа к существующим БД предоставляет

широкий спектр технологических решений, по-разному связанных между собой – перекрывающих, взаимодействующих и т.д. Выбор конкретных решений при обеспечении доступа зависит от специфики конкретной СУБД и от ряда других факторов, как то: наличие специалистов, способных с минимальными издержками освоить определенную ветвь технологических решений, существование других БД, Веб-доступ к которым должен осуществляться с минимальными дополнительными затратами и т.д.

Веб-доступ к существующим базам данных может осуществляться по одному из трех основных сценариев. Ниже дается их краткое описание и основные характеристики.

9.1.5.1. Преобразование содержимого БД в статические документы

В случае однократного или периодического преобразования содержимого БД в статические документы содержимое БД просматривает специальная программа, создающая множество файлов – связанных HTML-документов. Полученные файлы могут быть перенесены на один или несколько Веб-серверов. Доступ к ним будет осуществляться как к статическим гипертекстовым документам сервера.

Этот вариант характеризуется минимальными начальными расходами. Он эффективен на небольших массивах данных простой структуры и редким обновлением, а также при пониженных требованиях к актуальности данных, предоставляемых через Веб.

9.1.5.2. Динамическое создание гипертекстовых документов

В случае динамического создания гипертекстовых документов на основе содержимого БД доступ к БД осуществляется специальной программой (например, CGI-программой), запускаемой Веб-сервером в ответ на запрос Веб-клиента. Эта программа, обрабатывая запрос, просматривает содержимое БД и создает выходной HTML-документ, возвращаемый клиенту.

Это решение эффективно для больших баз данных со сложной структурой и при необходимости поддержки операций поиска. Показаниями также являются частое обновление и невозможность синхронизации преобразования БД в статические документы с обновлением содержимого. В этом варианте возможно осуществлять изменение БД из Веб-интерфейсов. К недостаткам этого метода можно отнести большое время обработки запросов, необходимость постоянного доступа к основной базе данных, дополнительную загрузку средств поддержки БД, связанную с обработкой запросов от Веб-сервера.

9.1.5.3. Создание информационного хранилища на основе СУБД

В случае создания информационного хранилища на основе высокопроизводительной СУБД с языком запросов SQL и периодической загрузки данных в хранилище из основных СУБД для обработки разнообразных запросов, в том числе и от Веб-сервера, используется промежуточная БД высокой производительности. Информационное наполнение промежуточной БД осуществляется специализированным программным обеспечением на основе содержимого основных баз данных.

Данный вариант свободен ото всех недостатков предыдущих схем. Более того, после установления синхронизации данных информационного хранилища с основными БД возможен перенос пользовательских интерфейсов на информационное хранилище, что существенно повысит надежность и производительность, позволит организовать распределенные рабочие места. Несмотря на кажущуюся громоздкость такой схемы, для задач обеспечения Веб-доступа к содержимому нескольких баз данных накладные расходы существенно уменьшаются.

9.1.6. Язык SQL

SQL является инструментом, предназначенным для обработки и чтения данных, содержащихся в компьютерной базе данных. SQL – это сокращенное название структурированного языка запросов (Structured Query Language) [26]. SQL основывается на реляционной алгебре.

Вопреки существующим заблуждениям, SQL в его чистом (базовом) виде является информационно-логическим языком, а не языком программирования [27]. Вместе с тем стандарт языка спецификацией SQL/PSM предусматривает возможность его процедурных расширений, с учетом которых язык уже вполне может рассматриваться в качестве языка программирования.

На рис. 9.4 изображена схема работы SQL



Рис. 9.4. Применение SQL для доступа к базе данных

SQL используется для реализации всех функциональных возможностей, которые СУБД предоставляет пользователю, а именно:

- Организация данных. SQL дает пользователю возможность изменять структуру представления данных, а также устанавливать отношения между элементами базы данных.
- Чтение данных. SQL дает пользователю или приложению возможность читать из базы данных содержащиеся в ней данные и пользоваться ими.
- Обработка данных. SQL дает пользователю или приложению возможность изменять базу данных, т.е. добавлять в нее новые данные, а также удалять или обновлять уже имеющиеся в ней данные.
- Управление доступом. С помощью SQL можно ограничить возможности пользователя по чтению и изменению данных и защитить их от несанкционированного доступа.
- Совместное использование данных. SQL координирует совместное использование данных пользователями, работающими параллельно, чтобы они не мешали друг другу.
- Целостность данных. SQL позволяет обеспечить целостность базы данных, защищая ее от разрушения из-за несогласованных изменений или отказа системы.

Таким образом, SQL является достаточно мощным языком для взаимодействия с СУБД.

Все операторы SQL в общем виде можно представить так, как это показано на рис. 9.5.



Рис. 9.5. Общий вид оператора SQL

Каждый оператор SQL начинается с глагола, представляющего собой ключевое слово, определяющее, что именно делает этот оператор (SELECT, INSERT, DELETE...). В операторе содержатся также предложения, содержащие сведения о том, над какими данными производятся операции. Каждое предложение начинается с ключевого слова, такого как FROM, WHERE и др. Структура предложения зависит от его типа – ряд предложений содержит имена таблиц или полей, некоторые могут содержать дополнительные ключевые слова, константы или выражения.

Язык SQL делится на несколько частей [27, 28]:

- Операторы определения данных (Data Definition Language, DDL, табл. 9.2);

Таблица 9.2. Операторы определения данных	
Оператор	Описание
CREATE TABLE	Применяется для добавления новой <i>таблицы</i> к базе данных
DROP TABLE	Применяется для удаления <i>таблицы</i> из базы данных
ALTER TABLE	Применяется для изменения структуры имеющейся <i>таблицы</i>
CREATE VIEW	Применяется для добавления нового представления к базе данных
DROP VIEW	Применяется для удаления представления из базы данных
CREATE INDEX	Применяется для создания индекса для данного поля
DROP INDEX	Применяется для удаления существующего индекса
CREATE SCHEMA	Применяется для создания новой схемы в базе данных
DROP SCHEMA	Применяется для удаления схемы из базы данных
CREATE DOMAIN	Применяется для создания нового домена
ALTER DOMAIN	Применяется для переопределения домена
DROP DOMAIN	Применяется для удаления домена из базы данных

- Операторы манипуляции данными (Data Manipulation Language, DML, табл. 9.3);

Таблица 9.3. Операторы манипуляции данными	
Оператор	Описание
SELECT	Применяется для выбора данных
INSERT	Применяется для добавления <i>строк</i> к <i>таблице</i>
DELETE	Применяется для удаления <i>строк</i> из <i>таблицы</i>
UPDATE	Применяется для изменения данных

- Операторы определения доступа к данным (Data Control Language, DCL, табл. 9.4);

Таблица 9.4. Операторы определения доступа к данным	
Оператор	Описание
GRANT	Применяется для присвоения привилегии

REVOKE	Применяется для отмены привилегии
--------	-----------------------------------

- операторы управления *транзакциями* (*Transaction Control Language, TCL*, табл. 9.5);

Таблица 9.5. Операторы управления <i>транзакциями</i>	
Оператор	Описание
COMMIT	Применяется для завершения <i>транзакции</i> и сохранения изменений в базе данных
ROLLBACK	Применяется для отката <i>транзакции</i> и отмены изменений в базе данных
SET TRANSACTION	Применяется для установки параметров доступа к данным в текущей <i>транзакции</i>

- операторы определения курсора (*Cursor Control Language, CCL*, табл. 9.6).

Таблица 9.6. Операторы определения курсора	
Оператор	Описание
DECLARE CURSOR	Применяется для определения курсора для запроса
EXPLAIN	Применяется для описания плана запроса. Этот оператор представляет собой расширение SQL для <i>Microsoft SQL Server 7.0</i> . Он не обязан выполняться в других СУБД. Например, в случае Oracle следует использовать оператор EXPLAIN PLAN
OPEN CURSOR	Применяется для открытия курсора при получении результатов запроса
FETCH	Применяется для получения <i>строки</i> из результатов запроса
CLOSE CURSOR	Применяется для закрытия курсора
PREPARE	Применяется для подготовки оператора SQL для выполнения
EXECUTE	Применяется для выполнения оператора SQL
DESCRIBE	Применяется для описания подготовленного запроса

Достоинства SQL [26, 27]:

- независимость от конкретных СУБД;
- переносимость с одной вычислительной системы на другую;
- наличие стандартов;
- реляционная основа;
- высокоуровневая структура, напоминающая английский язык;
- возможность выполнения специальных интерактивных запросов;
- обеспечение программного доступа к базам данных;
- возможность различного представления данных;
- полноценность как языка, предназначенного для работы с базами данных;
- возможность динамического определения данных;
- поддержка архитектуры клиент/сервер;
- декларативность.

Недостатки [27]:

- Несоответствие реляционной модели данных [29]:
- повторяющиеся строки;

- неопределенные значения (nulls);
- явное указание порядка колонок слева направо;
- колонки без имени и дублирующие имена колонок;
- отсутствие поддержки свойства "=";
- использование указателей;
- высокая избыточность;
- Сложность;
- Отступления от стандартов в конкретных реализациях СУБД;
- Сложность работы с иерархическими структурами.

Поскольку SQL не является языком программирования, вводимые разными производителями расширения касались в первую очередь процедурных расширений. Это хранимые процедуры (stored procedures) и процедурные языки-"надстройки". Практически в каждой СУБД применяется свой процедурный язык. Стандарт для процедурных расширений представлен спецификацией SQL/PSM). Перечень процедурных расширений для самых популярных СУБД приведен в табл. 9.7.

Таблица 9.7. Процедурные расширения популярных СУБД	
СУБД	Расширение
InterBase/Firebird	PSQL – Procedural SQL
IBM DB2	SQL PL – SQL Procedural Language (расширяет SQL/PSM)
MS SQL Server/ Sybase ASE	Transact-SQL
MySQL	SQL/PSM – SQL/Persistent Stored Module (соответствует стандарту SQL:2003)
Oracle	PL/SQL – Procedural Language/SQL (основан на языке Ada)
PostgreSQL	PL/pgSQL – Procedural Language/PostgreSQL Structured Query Language (очень похож на Oracle PL/SQL)

9.1.7. Ключевые термины

База данных, Система управления базами данных, Модель данных, Иерархическая модель данных, Сетевая модель данных, Реляционная модель данных, Постреляционная модель данных, Объектно-ориентированная модель данных, Объектно-реляционная модель данных, Таблицы, Столбы, Строки, Первичный ключ, Внешний ключ, Индексирование, Нормальная форма, Нормализация, Транзакция, ACID, Проектирование баз данных, SQL.

9.2. Доступ к данным в .NET

В состав платформы .NET Framework входят функции, позволяющие добавить доступ к данным на Веб-страницах ASP.NET с написанием минимально необходимого программного кода или вовсе без него. В качестве источников данных можно использовать базы данных, данные и файлы XML, а также бизнес-объекты. Затем данные можно отобразить на странице с помощью разнообразных элементов управления, обеспечивающих широкие возможности представления данных на странице.

В качестве основных технологий .NET для доступа к данным выступают:

- ADO.NET – основная модель доступа к данным для приложений, основанных на Microsoft .NET.
- Linq – проект Microsoft по добавлению синтаксиса языка запросов, напоминающего SQL, в языки программирования платформы .NET Framework 3.5.

- ADO.NET Entity Framework – объектно-ориентированная технология доступа к данным, является ORM-решением для .NET Framework 3.5.

В данной лекции рассмотрим технологию ADO.NET.

9.2.1. СУБД MS SQL Server 2008

Прежде чем рассматривать технологии доступа к данным рассмотрим СУБД Microsoft SQL Server 2008, которая как раз и позволяет хранить данные и управлять базами данных.

Microsoft SQL Server – система управления реляционными базами данных (СУБД), разработанная корпорацией Microsoft. Основным используемым языком запросов – Transact-SQL, создан совместно Microsoft и Sybase. Transact-SQL является реализацией стандарта ANSI/ISO по структурированному языку запросов (SQL) с расширениями.

Так как SQL Server 2008 расширяет и улучшает функции SQL Server 2005, а не является революционным продуктом, то характеристики этой СУБД начнем рассматривать с 2005-ой версии.

9.2.1.1. SQL Server 2005

Microsoft SQL Server 2005 представляет собой платформу для работы с базами данных, обеспечивающую возможность крупномасштабной оперативной обработки транзакций (OLTP), хранения данных и работы с приложениями для электронной торговли; а также является платформой бизнес-аналитики для создания решений по интеграции данных, анализу и составлению отчетов [30].

На рис. 9.6 представлены связи и взаимодействия между компонентами SQL Server 2005.

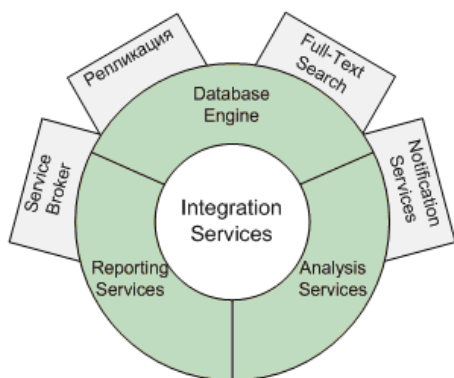


Рис. 9.6. Компоненты SQL Server 2005

SQL Server 2005 представляет "среды" для облегчения выполнения задач разработки и управления: среда SQL Server Management Studio и среда Business Intelligence Development Studio. В среде Management Studio можно разрабатывать решения компонента SQL Server Database Engine и решения по уведомлению и управлять ими, управлять развернутыми решениями служб Analysis Services, запускать пакеты служб Integration Services и управлять ими, управлять серверами, отчетами и моделями отчетов служб Reporting Services. В среде BI Development Studio можно разрабатывать решения бизнес-аналитики: проекты служб Analysis Services используются для разработки кубов, измерений и структур интеллектуального анализа данных; проекты служб Reporting Services – для создания отчетов; проекты моделей отчетов – для определения моделей отчетов и проекты служб Integration Services – для создания пакетов.

Рассмотрим подробнее компоненты, изображенные на рис. 9.9.

[illegible]

Службы Analysis Services являются основными службами по обеспечению быстрого анализа бизнес-данных, интерактивной аналитической обработки (OLAP) и функций интеллектуального анализа данных для использования в приложениях бизнес-аналитики.

Службы Integration Services – это платформа для построения решений по интеграции и преобразованию данных уровня предприятия. Службы Integration Services используются для решения сложных бизнес-задач при помощи копирования и загрузки файлов, отправки электронных сообщений в ответ на события, обновления хранилищ данных, очистки и интеллектуального анализа данных, а также управления объектами и данными SQL Server. Пакеты могут работать отдельно или совместно с другими пакетами для решения сложных бизнес-задач. Службы Integration Services могут извлекать и преобразовывать данные из ряда источников, таких как файлы данных XML, плоские файлы,

источники реляционных данных, и затем загружать эти данные в один или несколько реляционных объектов.

Репликация представляет собой набор технологий, с помощью которых данные или объекты баз данных можно скопировать и перенести из одной базы данных в другую, а затем синхронизировать эти базы данных для обеспечения согласованности. Используя репликацию, можно распространять данные через локальные или глобальные сети, соединения по телефонной линии, беспроводные соединения и Интернет для их размещения в различных точках или для удаленных и мобильных пользователей. SQL Server предусматривает три типа репликации, отличающихся по своим возможностям: это репликация транзакций, репликация слиянием и репликация моментальных снимков.

Службы Notification Services – это платформа для разработки приложений, формирующих и отправляющих уведомления; кроме того, это ядро, запускающее такие приложения. Службы Notification Services могут формировать и посылать периодические, персонализированные сообщения тысячам и даже миллионам подписчиков и доставлять сообщения в разнообразные приложения и на различные устройства.

SQL Server включает функции, необходимые для выполнения полнотекстовых запросов к неформатированным символьным данным в таблицах SQL Server. Полнотекстовые запросы могут включать слова и фразы или несколько форм слов или фраз. Полнотекстовый поиск обеспечивает быстрое и гибкое индексирование для основанных на ключевых словах запросов текстовых данных, хранящихся в базе данных Microsoft SQL Server.

Компонент Service Broker обеспечивает собственную поддержку компонента SQL Server Database Engine для приложений обмена сообщениями и приложений с очередями сообщений. Это облегчает разработчикам создание сложных приложений, использующих компоненты Database Engine для связи между разнородными базами данных. Разработчики могут использовать компонент Service Broker для облегчения создания распределенных и надежных приложений.

9.2.1.2. Новинки SQL Server 2008

Концептуальная модель SQL Server 2008 представлена на рис. 9.8.



Рис. 9.8. SQL Server 2008

Источник: Microsoft SQL Server 2008 [31]

Возможности SQL Server 2008 относятся к четырем основным направлениям развития представлений Microsoft о платформе данных [31, 32]:

- Платформа для критических приложений (Mission Critical Platform):
 - Шифрование базы данных (БД), файлов данных или файлов журналов.
 - Поддержка HSM (Hardware Security Module, аппаратный модуль безопасности) и системы управления ключами сторонних производителей.
 - Создание пользовательских параметров аудита и управление ими, используя DDL-операторы (Data Definition Language, язык определения данных).
 - Обеспечение сжатия исходящего потока данных для журналов.
 - Возможность "горячей" установки процессоров.
 - Среда декларативного управления Declarative Management Framework (DMF)
 - Расширенный набор показателей производительности, новое централизованное хранилище данных о работоспособности и новые инструменты для составления отчетов и мониторинга.
 - Компонент Resource Governor позволяет устанавливать ограничения на использование ресурсов и задавать приоритеты для различных задач.
- Динамическая разработка (Dynamic Development):
 - Среда ADO.NET Entity Framework.
 - Встроенные запросы LINQ (Language Integrated Query).
 - Уровень ADO.NET-сервисов Object.
 - Нерегулярно подключающиеся системы.
 - Новые типы нереляционных данных.
- Хранение данных любых типов (Beyond Relational Data).
 - Новые типы данных для обозначения даты и времени:
 - **DATE** – только дата;
 - **TIME** – только время;
 - **DATETIMEOFFSET** – дата и время с учетом часового пояса;
 - **DATETIME2** – тип для даты и времени с поддержкой большего диапазона долей секунд и лет, чем в существующем типе DATETIME.
 - Системный тип HierarchyId, предназначенный для хранения значений, представляющих узлы иерархии.
 - Тип данных FILESTREAM, обеспечивающий хранение больших объемов двоичных данных непосредственно в файловой системе NTFS.
 - Компонент "Разреженные поля", с помощью которого значения NULL не занимают физическое пространство.
 - Нет 8-килобайтного ограничения для пользовательских типов (User Defined Type, UDT).
 - Использование данных о географическом расположении и местонахождении.
 - Векторное решение для пространственных данных, согласующееся с промышленными стандартами.
- Всесторонне обоснованные решения (Pervasive Business Insight).
 - Расширены возможности секционирования.
 - Повышена производительность запросов в наиболее типовых сценариях работы с хранилищами данных (в частности, оптимизация Star Join).
 - Grouping Sets – расширенная версия раздела GROUP BY.
 - Функция контроля за изменением данных.

- Введение SQL-оператора MERGE.
- Усовершенствования в Integration Services:
- Улучшения в конвейере SQL Server Integration Services (SSIS).
- Просмотры SSIS с сохранением.
- Блочные вычисления.
- Обратная запись MOLAP в Analysis Services.
- Усовершенствованный конструктор отчетов Report Designer.
- Встроенная аутентификация форм.
- Более тесное взаимодействие SQL Server 2008 с Microsoft Office.

Начиная с версии SQL Server 2005, SQL Server обеспечивает интеграцию с компонентами CLR (Common Language Runtime, общезыковая исполняющая среда) платформы .NET Framework для Microsoft Windows [33]. Это означает, что хранимые процедуры, триггеры, определяемые пользователем типы, определяемые пользователем функции, определяемые пользователем статистические функции и возвращающие табличные значение потоковые функции теперь могут разрабатываться с использованием любого языка .NET Framework, включая Microsoft Visual Basic .NET и Microsoft Visual C#.

9.2.2. ADO.NET

9.2.2.1. Общие сведения

ADO.NET представляет собой набор библиотек, входящих в Microsoft .NET Framework предназначенных для взаимодействия с различными хранилищами данных из .NET приложений [34-37]. Библиотеки ADO.NET включают все необходимые классы для подключения к источникам данных практически произвольного формата, выполнения запросов к этим источникам и получения результата. Кроме того, несомненным достоинством ADO.NET является возможность работы с отсоединенными источниками данных, представляющих собой структуры, организующие данные в оперативной памяти компьютера, работать с которыми возможно с использованием ставших уже привычными средств доступа к данным.

Так как ADO.NET представляет собой набор классов для организации взаимодействия клиентского приложения с базой данных, рассмотрим объектную модель ADO.NET (рис. 9.9).

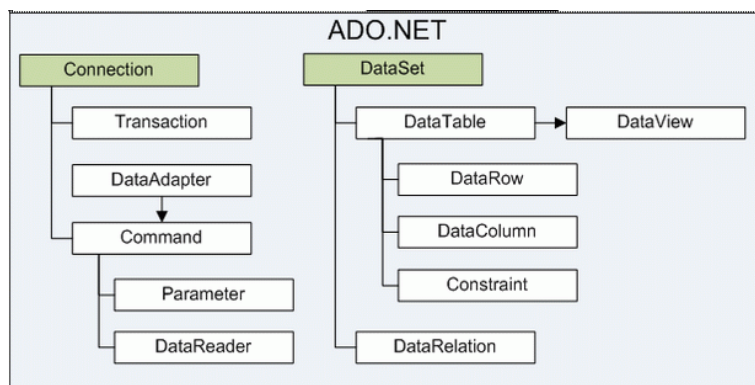


Рис. 9.9. Иерархия объектов ADO.NET

Объекты, расположенные в левой части называются подсоединенными и необходимы для управления соединением, транзакциями, выборкой данных и передачей изменений данных в БД, т.е. они непосредственно взаимодействуют с базой данных.

- Объект Connection представляет соединение с источником данных.
- Объект Transaction позволяет осуществлять транзакции.

- Объект `DataAdapter` представляет собой связующее звено между отсоединенными объектами ADO.NET и базой данных. С его помощью осуществляется заполнение таких объектов как `DataSet` или `DataTable` значениями, полученными в результате выполнения запроса к базе данных для последующей автономной работы с ними. Помимо этого, `DataAdapter` реализует эффективный механизм выполнения обновления данных, хранимых в базе данных изменениями, внесенными в данные объектов `DataSet` и `DataTable`.
- Объект `Command` представляет запрос к источнику данных, вызов хранимой процедуры или прямой запрос на возврат содержимого конкретной таблицы.
- Объект `Parameter` позволяет вводить в запрос элемент, значение которого может быть задано непосредственно перед исполнением запроса.
- Объект `DataReader` предназначен для максимально быстрой выборки и просмотра возвращаемых запросом записей.
- Объекты, расположенные в правой части называются отсоединенными и позволяют работать с данными автономно.
- Объект `DataSet` представляет собой отсоединенный набор данных, который может рассматриваться как контейнер для объектов `DataTable`. `DataSet` позволяет организовывать внутри себя структуру, полностью соответствующую реальной структуре таблиц и связей между ними в БД.
- Объект `DataTable` позволяет просматривать данные в виде наборов записей и столбцов.
- Объект `DataColumn` представляет собой столбец объекта `DataTable`. Набор же всех столбцов объекта `DataTable` представляет собой коллекцию `Columns`.
- Объект `DataRow` представляет собой строку объекта `DataTable`. Набор всех строк этого объекта представляет собой коллекцию `Rows`.
- Объект `DataView` предназначен для организации возможности просмотра содержимого `DataTable` различными способами. Это относится к таким операциям, как сортировка и фильтрация записей.
- Объект `DataRelation` представляет собой описание связей между таблицами реляционной базы данных. Он предоставляется объектом `DataSet` и позволяет организовывать взаимосвязи между таблицами отсоединенного набора данных объекта `DataSet`.

Одной из основных идей, лежащих в основе ADO.NET является наличие поставщиков данных.

Поставщик данных – это набор классов, предназначенных для взаимодействия с хранилищем данных определенного типа. Рассмотрим уровни модели поставщиков ADO.NET (рис. 9.10).

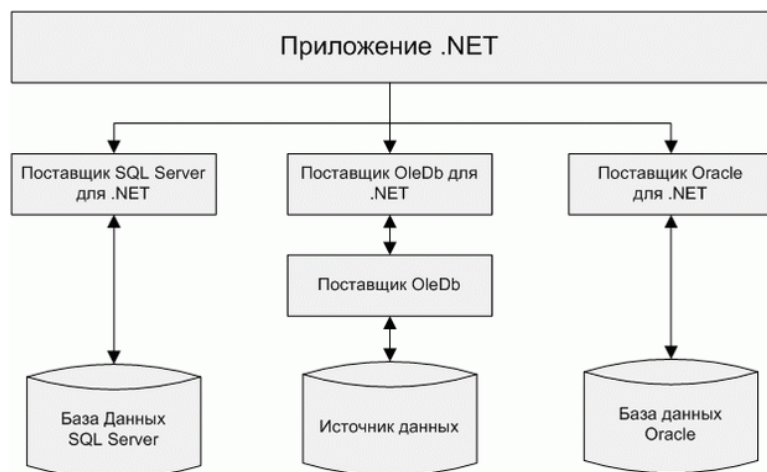


Рис. 9.10. Уровни моделей поставщиков ADO.NET

Как видно из рис. 9.10, приложение .NET взаимодействует с базой данных посредством поставщиков данных. Каждый поставщик данных может обеспечивать доступ только к базе данных определенного формата. Так, для доступа к БД Microsoft SQL Server используется поставщик SQL Server для .NET, для доступа к БД Oracle – поставщик Oracle для .NET и т.д. Если поставщики данных для каких-либо СУБД не выделены в ADO.NET, то для взаимодействия с ними может применяться поставщик данных OleDb для .NET или ODBC для .NET, которые обеспечивают доступ к любым данным, для которых существует драйвер OleDb, либо ODBC соответственно. Тем не менее, всегда предпочтительнее использовать тот поставщик данных, который специально предназначен для обеспечения доступа к данному источнику данных, т.к. он учитывает его особенности.

Каждый поставщик .NET реализует одинаковые базовые классы – Connection, Transaction, DataAdapter, Command, Parameter, DataReader имена которых зависят от поставщика. Например, у поставщика SQL Server существует объект SqlDataAdapter, у поставщика OleDb – OleDbDataAdapter и т.д.

У каждого поставщика данных существует собственное пространство имен. Хотя все поставщики относятся к пространству имен System.Data, каждый из них содержит свой подраздел этого пространства, который содержит объекты, специфичные для данного поставщика. Например, объект SqlDataAdapter находится в пространстве имен System.Data.SqlClient.

Все поставщики данных .NET реализуют одинаковые базовые функции, поэтому код, создаваемый для доступа к данным, выглядит приблизительно одинаково независимо от поставщика.

9.2.2.2. Организация взаимодействия с БД

Для того чтобы приложение .NET могло осуществлять взаимодействие с источником данных, необходимо установить соединение с ним. Наиболее типичным сценарием работы Веб-приложения с БД является следующий:

Устанавливается соединение, открывается подключение к базе данных.

Выполняется один или несколько запросов, осуществляющих внесение изменений в наборы данных источника данных, а также выборку данных из БД.

Осуществляется отключение от источника данных. При этом пользователь работает с отсоединенным набором данных, просматривая его, выполняя фильтрацию, внося изменения и т.д.

При необходимости переноса изменений, из отсоединенного набора данных в БД, а также при необходимости просмотра изменений, внесенных в БД другими пользователями, осуществляется подключение к источнику данных, выполняются необходимые действия, после чего производится отключение от БД.

Рассмотрим описанные выше этапы более подробно.

9.2.2.2.1. Подключение к БД

Для того чтобы подключиться к базе данных во время выполнения программы, необходимо создать объект Connection, а также задать его свойства, определяющие текущие параметры подключения. Основным параметром, устанавливающим необходимые опции для подключения к БД, является строка соединения, которая представляет собой набор пар "имя-значение", разделенных точкой с запятой. Строка соединения зависит от СУБД, к которой осуществляется подключение, а также от используемого поставщика данных.

Тем не менее, существует несколько фрагментов информации, указываемой в строке подключения, необходимых практически всегда:

- Адрес сервера, на котором находится база данных.
- Имя базы данных, к которой производится подключение.
- Способ аутентификации пользователя (например, имя пользователя и пароль для проверки возможности доступа к базе данных).

Дальнейшие примеры кода рассмотрены применимо к СУБД SQL Server 2008 Express.

Для подключения к БД используется объект Connection из пространства имен System.Data.SqlClient. Строка соединения с базой данных при этом будет выглядеть следующим образом:

```
string strSqlConnection = "Data Source=localhost\\sqlexpress;Initial  
Catalog=FORTEST;Integrated Security=SSPI";
```

где

Data Source=localhost\\sqlexpress – указывает, что подключаемся к localhost;

Initial Catalog=FORTEST – указывает, что имя базы данных FORTEST;

Integrated Security=SSPI – указывает на windows-авторизацию к SQL Server.

Для открытия соединения с базой данных необходимо вызвать метод Open объекта Connection. При этом, строку соединения с БД можно передать как в качестве параметра конструктора объекта, так и потом с помощью установки соответствующего свойства:

```
SqlConnection sqlCon=new SqlConnection(strSqlConnection);  
sqlCon.Open();
```

или

```
SqlConnection sqlCon=new SqlConnection();  
sqlCon.ConnectionString= strSqlConnection;  
sqlCon.Open();
```

Строку соединения с базой данных можно жестко прописать в исходном коде приложения, однако, это не самый лучший вариант, т.к. при изменении пути к базе данных, либо других параметров соединения придется вносить изменения в исходный код приложения и перекомпилировать его. В связи с этим, лучше всего использовать строку соединения, сохраненную в файле web.config.

```
<connectionStrings>  
  <add name="FORTEST_ConnectionString"  
        connectionString="Provider= System.Data.SqlClient;Data  
                          Source=C:\FORTEST.mdb"/>  
</connectionStrings>
```

Впоследствии эту строку можно извлечь из файла web.config:

```
string strSqlConnection = WebConfigurationManager.ConnectionStrings  
["FORTEST_ConnectionString"].ConnectionString;
```

Управление соединением осуществляется с помощью методов Open и Close объекта Connection. Однако следует учитывать, что при подключении к базе данных может произойти сбой, в результате которого установить соединение с

ней окажется невозможно. Поэтому рекомендуется использовать конструкции try catch, позволяющие адекватно реагировать на возникшую ошибку:

```
try
{
    sqlCon.Open();
}
catch(Exception ex)
{
    return ex.Message;
}
```

Для обслуживания соединения с базой данных расходуются ресурсы компьютера, на котором выполняется Веб-приложение. В связи с этим, рекомендуется открывать соединение как можно позже, а закрывать как можно раньше, сразу после того, как все необходимые действия с объектами базы данных были выполнены. Кроме того, желательно строить программный код так, чтобы соединение с базой данных закрывалось при любом исходе соединения с ней (т.е. выполнялся метод Close()).

9.2.2.2.2. Выполнение команд над наборами данных

Одним из основных элементов из набора классов ADO.NET, способным выполнять любой SQL оператор является класс Command. Для того чтобы использовать класс Command необходимо установить тип команды, установить текст запроса SQL и привязать ее к соединению с БД.

Существует 3 типа команд класса Command (табл. 9.8).

Таблица 9.8. Типы команд класса Command	
Тип команды	Описание
CommandType.Text	Выполнение прямого оператора SQL, текст которого устанавливается в свойстве CommandText . Это значение по умолчанию.
CommandType.StoredProcedure	Выполнение хранимой процедуры, имя которой установлено в свойстве CommandText .
CommandType.TableDirect	Выполняет опрос всей <i>таблицы</i> базы данных, имя которой задается в свойстве CommandText . Этот тип команды используется для обратной совместимости с некоторыми драйверами OLE DB и не поддерживается поставщиком данных SQL Server.

Пример создания объекта **Command** для выполнения SQL запроса:

```
SqlCommand command_SQL = new SqlCommand("Select * From  
FORTEST_Table",sqlCon);  
command_SQL.CommandType = CommandType.Text;
```

Для выполнения созданной команды необходимо использовать один из следующих методов (табл. 9.9).

Таблица 9.9. Методы класса Command	
Метод	Описание
ExecuteReader()	Выполнение запроса SQL и возврат объекта DataReader , представляющего однонаправленный курсор, с доступом только для чтения.
ExecuteNonQuery()	Выполнение SQL команд, предназначенных для вставки,

	изменения, удаления записей БД. Результатом работы команды является количество строк, обработанных командой.
ExecuteScalar()	Выполнение SQL команды и возврат первой строки результата запроса. Обычно используется для выполнения команды, содержащей агрегирующие функции типа COUNT() , MAX() и т.д.

Рассмотрим подробнее метод **ExecuteReader()**.

При использовании метода **ExecuteReader()** создается объект **DataReader**, с помощью которого можно организовать перебор всех строк возвращенного набора данных. Для этого необходимо использовать цикл по строкам результирующего набора данных:

```
SqlDataReader reader_SQL = command_SQL.ExecuteReader();
while (reader_SQL.Read())
{
    //берем данные из rdr_SQL
}
reader_SQL.Close();
```

Обращение к полям таблицы может происходить как по имени, так и с помощью индекса. В случае с индексом, необходимо применять метод, соответствующий типу данных того поля, к которому происходит обращение.

Использованный подход достаточно трудоемок для того, чтобы его применять на практике для вывода информации, содержащейся в базе данных. В ASP.NET существует несколько более мощных классов, способных производить данную операцию. Одним из таких классов является класс **GridView**. Применение данного класса позволяет осуществлять вывод информации на экран очень простым способом. В следующем примере показан способ вывода информации, получаемой из БД посредством объекта **DataReader**:

```
GridView1.DataSource = reader_SQL;
GridView1.DataBind();
```

9.2.2.2.3. Использование параметризованных команд

Параметризованная команда – это обычная SQL команда, в тексте которой используются специальные символы, указывающие место для динамической подстановки значений, передаваемых объекту **Command** через коллекцию **Parameters**. Например, команда удаления записи из таблицы будет выглядеть следующим образом при использовании параметра.

```
string strSQLServer = "DELETE FROM FORTEST_Table WHERE Column1=@Parameter";
```

Здесь **@Parameter** представляет собой параметр, значение которого должно быть установлено до того, как будет запущено выполнение запроса. Синтаксис параметризованных команд отличается в разных поставщиках данных. Приведенный выше пример справедлив для взаимодействия с **SQL Server**. Для означивания параметра можно выполнить, например, следующий код:

```
SqlCommand cmdDeleteSQLServer = new SqlCommand(strSQLServer,sqlCon);
cmdDeleteSQLServer.Parameters.Add("@ProductName", "Text");
```

Кроме того, что работа с параметризованными запросами проще с точки зрения динамического формирования текста самого запроса, она еще обеспечивает защиту от атаки внедрением SQL.

9.2.2.2.4. Использование хранимых процедур

Хранимая процедура представляет собой набор операторов SQL, сохраненный в базе данных (на стороне сервера), имеющий имя, а также набор параметров, посредством которых могут быть переданы значения в хранимую процедуру. Они подобны процедурам, используемых в языках программирования с той лишь разницей, что в данном случае, предназначены для обработки данных, содержащихся в базе данных.

Хранимые процедуры обладают рядом преимуществ, главными из которых являются такие, как повышение быстродействия приложения при использовании хранимых процедур для обработки данных, повышение безопасности приложения при работе с данными.

Рассмотрим пример создания и использования простой хранимой процедуры, позволяющей добавить новую запись в таблицу:

```
CREATE PROCEDURE AddProduct
@Column1 int,
    @Column2 varchar(100)
AS
INSERT INTO FORTEST_Table
VALUES(@Column1,@ Column2)
```

Данная хранимая процедура использует два значения переменных для формирования запроса на добавление данных в таблицу.

Для ее вызова можно воспользоваться объектом SqlCommand. Текст программы, реализующей вызов данной хранимой процедуры, приведен ниже.

```
SqlCommand command_SQL = new SqlCommand("AddRows",sqlCon);
command_SQL.CommandType = CommandType.StoredProcedure;
string[] strProduct = tb_AddProduct.Text.Split(new char[] {';'});
command_SQL.Parameters.Add(new SqlParameter("@Column1", SqlDbType.Int,
4));
command_SQL.Parameters["@Column1"].Value = Convert.ToInt32(strProduct[0]);
command_SQL.Parameters.Add(new SqlParameter("@Column2",
SqlDbType.NVarChar, 100));
command_SQL.Parameters["@Column2"].Value = strProduct[1];
try
{
    sqlCon.Open();
    int k = cmd_SQL.ExecuteNonQuery();
}
finally
{
    sqlCon.Close();
}
```

В процессе работы данной программы, введенный пользователем текст в элемент TextBox преобразуется в массив строк. В коллекцию Parameters объекта SqlConnection добавляется два параметра, соответствующих параметрам хранимой процедуры. Для них устанавливаются значения, ранее помещенные в массив строк. После чего открывается соединение и посылается запрос на исполнение хранимой процедуры посредством команды ExecuteNonQuery().

9.2.2.3. Отсоединенные наборы данных

ADO.NET предоставляет более совершенные способы организации двустороннего взаимодействия приложения с базой данных, основанной на отсоединенных наборах данных.

Основной идеей использования отсоединенных наборов данных является изменения алгоритмов взаимодействия приложения с базой данных за счет подключения к набору данных, выполнения запроса и создания копии данных на стороне клиента, отключения от БД, осуществление манипуляций с данными на стороне клиента, при необходимости внесения изменений в базу данных, подключение к ней, передача изменений и отключение. Таким образом, все основные манипуляции с данными происходят в отсоединенной наборе данных, представляющем собой копию данных, хранящихся в БД, а внесение изменений происходит в одной пакетной операции. Все это уменьшает время, в течение которого должно быть открыто соединение с БД, ускоряет работу и упрощает логику взаимодействия приложения с данными. Рассмотрим объекты ADO.NET, реализующие данный механизм.

9.2.2.3.1. Класс DataSet

Объект DataSet представляет собой контейнер, содержащий объекты DataTable и Relation. DataTable, представляет собой таблицу, состоящую из строк и столбцов. Строки таблицы представлены объектом DataRow, который, в свою очередь, представляет собой коллекцию столбцов таблицы (объект DataColumn). Данные в DataSet отсоединены от БД. Все изменения данных кэшируются в объектах DataRow. При возникновении необходимости передачи изменений в данных объекта DataSet существует возможность передачи только изменившейся части данных, что позволяет значительно экономить ресурсы канала связи, т.к. в этом случае передается гораздо меньший объем данных. Обобщая все вышесказанное можно сделать вывод о том, что использование объекта DataSet в ряде случаев оказывается более эффективным, чем DataReader. Наиболее типичными ситуациями, в которых рекомендуется применять объект DataSet, являются:

- Необходимость реализации сериализации данных на диск. DataSet позволяет легко сохранять данные в файле XML. При этом возможны варианты сохранения только данных, только структуры данных, либо и того и другого.
- Необходимость организации навигации по набору данных в двух направлениях. Как уже упоминалось, DataReader обеспечивает перемещение по набору только вперед. С помощью же DataSet возможна организация страничного просмотра данных.
- Необходима привязка нескольких элементов управления к одному набору данных. DataSet, в отличие от DataReader содержит средства организации сортировки и фильтрации данных.

Для извлечения данных из базы данных и наполнения ими объекта DataSet необходимо использовать еще один объект DataAdapter, который помимо прочего позволяет обновлять данные БД на основе внесенных в DataSet изменений.

9.2.2.3.2. Класс DataAdapter

DataAdapter является связующим звеном между базой данных и DataSet. Точнее, он связывает БД и объект DataTable, расположенный внутри DataSet.

DataAdapter содержит три основных метода, позволяющих ему выполнять все необходимые операции, связанные с извлечением и обновлением данных (табл. 9.10).

Таблица 9.10. Методы класса DataAdapter	
Метод	Описание
Fill	Выполнение запроса типа Select , определенного в свойстве SelectCommand и добавление <i>таблицы</i> , получаемой в результате данного запроса в DataSet .
FillSchema()	Выполнение запроса типа Select , текст которого расположен в свойстве SelectCommand и добавление <i>таблицы</i> , содержащей только структуру данных полученных в результате выполнения запроса в DataSet .
Update()	Применяет все изменения, внесенные в DataTable к источнику данных. При этом исполняются команды вставки, обновления и удаления, расположенные в свойствах InsertCommand , UpdateCommand , DeleteCommand .

Рассмотрим пример использования объектов DataSet и DataAdapter для извлечения данных из БД, наполнения DataSet и отображения данных на странице Веб-приложения.

Прежде всего, необходимо установить подключение к источнику данных. Для этого необходимо использовать строку подключения, а также объект Connection. Воспользуемся предыдущими примерами:

```
string strSqlConnection = "Data Source=localhost\\sqlexpress;Initial
Catalog=FORTEST;Integrated Security=SSPI";
string sqlString = "Select * From FORTEST_Table";
SqlConnection sqlCon=new SqlConnection(strSqlConnection);
```

Создадим объект DataAdapter и передадим ему в качестве параметров строку запроса, а также строку подключения к БД.

```
SqlDataAdapter da = new SqlDataAdapter(sqlString,sqlCon);
```

Теперь необходимо создать объект DataSet и заполнить его данными с помощью DataAdapter.

```
DataSet ds = new DataSet();
da.Fill(ds, "Records");
```

Из приведенного примера видно, что метод Fill объекта da, в качестве параметров использует имя объекта DataSet, в который необходимо поместить данные, возвращаемые в результате выполнения запроса, определенного в строке sqlString. Во втором параметре можно указать имя, которое будет сопоставлено с таблицей, созданной в DataSet. Из примера видно, что в данном случае не используется явный вызов метода Open объекта Connection. Это объясняется тем, что данный метод вызывается неявно при исполнении метода Fill. Таким образом, DataAdapter сперва открывает соединение с БД, затем выполняет необходимые манипуляции с данными, после чего закрывает открытое ранее соединение.

9.2.2.4. Привязка и отображение данных

Большинство элементов управления ASP.NET поддерживают привязку данных. При этом привязка данных может быть как данных с одним значением, так и с множественными значениями. Привязка с одним значением означает, что элемент управления может отображать единственное значение, извлекаемое из источника данных. Такие принципы используются элементами управления TextBox, LinkButton, Image, HyperLink. Привязка с множественными значениями означает, что элемент управления может отображать несколько значений, извлекаемых из источника данных. Элементы управления, поддерживающие привязку с множественными значениями, строятся на основе списков и электронных таблиц. Типичными представителями таких элементов управления являются ListBox и GridView.

Для привязки элементов управления к источнику данных необходимо установить значение свойства DataSource, в котором указать наименование объекта, содержащего данные, необходимые для отображения. При установке свойства DataSource создается логическая связь между элементом управления, способным отображать данные и объектом данных, подлежащих отображению. После того, как источник данных определен, необходимо заполнить его данными. Для этого используется метод DataBind() элемента управления, заполняемого данными, который извлекает данные из источника, проходит в цикле по источнику, определенному в DataSource и обновляет страницу. Привязка с множественными значениями является наиболее мощным типом привязки, так как в этом случае не требуется программировать циклический перебор значений источника данных и вывод их на экран, эта логика уже реализована в элементе управления, поддерживающего множественную привязку.

Рассмотрим основные принципы использования обоих типов привязки данных.

9.2.2.4.1. Привязка с одним значением

Элементы управления, поддерживающие привязку данных с одним значением, позволяют осуществить привязку некоторых из их свойств к данным с помощью выражения привязки данных. Выражение привязки данных вводится в тексте страницы .aspx и заключается внутри ограничителей <%# %>.

В качестве выражения привязки данных может быть использована общедоступная, либо защищенная переменная, а также любое другое выражение, которое может быть вычислено в момент выполнения страницы. Так, в качестве выражения привязки данных допустимо использовать функции, свойства, объекты, определенные в классе страницы.

Для того чтобы вычислить выражение привязки, необходимо в коде приложения вызвать метод Page.DataBind(). В момент вызова данного метода, ASP.NET проверяет все выражения на текущей странице, при необходимости производит вычисления и заменяет их соответствующими значениями.

В качестве примера рассмотрим привязку элемента Label к данным таблицы FORTEST_Table. Для этого создадим метод GetRecord(), возвращающий в качестве значения содержимое первой строки столбца Column1 таблицы FORTEST_Table:

```
public string GetRecord()
{
    return ds.Tables["Records"].Rows[0]["Column1"].ToString();
}
```

Теперь необходимо добавить элемент Label на страницу .aspx и установить выражение привязки для свойства Text данного элемента:

```
<asp:Label ID="Label1" runat="server"
Text="<%#GetRecord()%>"></asp:Label>
```

И в методе Page.Load() произведем вызов метода Page.DataBind().

Как видно из данного примера, таким способом можно отобразить только одно значение какого-либо поля таблицы.

9.2.2.4.2. Привязка с множественным значением

Для отображения набора значений необходимо использовать привязку с множественным значением. Привязываемый набор значений должен содержаться в объекте, способном хранить коллекции значений (например, объекты ArrayList, Hashtable и др.).

Каждый элемент, поддерживающий возможность множественной привязки, содержит следующий набор свойств (табл. 9.11).

Таблица 9.11. Свойства элементов, поддерживающих множественную привязку	
Свойство	Описание
DataSource	Объект, содержащий привязываемые данные.
DataSourceID	Свойство, содержащее идентификатор объекта, использующегося для подключения к источнику данных. В отличие от DataSource , значение DataSourceID можно установить в режиме дизайна на этапе разработки приложения.
DataTextField	Содержит наименование столбца (для строки) таблицы или свойство (для объекта) элемента данных, содержащего отображаемое на экране значение.
DataTextFormatString	Содержит необязательную строку форматирования, применяемую для отображения данных.
DataValueField	Содержит уникальный идентификатор или поле <i>первичного ключа</i> , используемое для извлечения данных, связанных с выбираемым пользователем элементом.

Принципы использования всех списковых элементов, способных отображать связанные данные, одинаковы. Для привязки такого элемента к источнику данных, необходимо установить значения свойств DataSource и DataTextField.

В следующем примере на странице размещен элемент DropDownList1, который связан с источником данных, в качестве которого выступает таблица FORTEST_Table.

```
DropDownList1.DataSource = ds.Tables["Columns"];
DropDownList1.DataTextField = "Column1";
DropDownList1.DataValueField = "Column2";
```

И в методе Page.Load() произведем вызов метода Page.DataBind().

Так как объект DataSet может содержать много таблиц, в свойстве DataSource необходимо указать имя таблицы, к которой необходимо осуществить привязку. Свойство DataTextField содержит наименование столбца таблицы, значения которого будут отображаться элементом управления. Свойство DataValueField является необязательным и, в данном случае, обозначает то, что при выборе значения в элементе управления, значение свойства SelectedItem этого элемента будет установлено равным значению поля Column2, соответствующего выбранной записи.

Кроме использованных списковых элементов, ASP.NET содержит также и более сложные элементы, способные привязываться к данным и отображать их. К таким элементам относятся GridView, DetailsView, FormView:

- GridView представляет собой таблицу, способную привязываться к данным и отображать содержимое таблиц. Поддерживает выбор, редактирование, сортировку и перемещение по страницам данных.
- DetailsView предназначен для показа одной записи из таблицы за один раз. Поддерживает редактирование и перемещение по страницам.
- FormView практически аналогичен DetailsView с той лишь разницей, что FormView основан на шаблонах, что делает его более гибким при отображении информации.

Самым мощным элементом по количеству возможностей является GridView. Очень часто его приходится использовать для отображения информации в виде таблицы. Для привязки GridView к источнику данных необходимо установить значение свойства DataSource. В следующем примере на странице размещен объект GridView, который привязывается к таблице FORTEST_Table и отображает ее содержимое.

```
GridView1.DataSource = ds.Tables["Columns"];
```

Показанные примеры демонстрируют способы привязки к источникам данных с использованием программного кода, однако это не единственная возможность осуществить привязку элемента управления к источнику данных. Visual Studio 2005/2008 позволяет осуществить привязку с использованием визуальных средств. Для этих целей .NET Framework содержит несколько элементов управления источниками данных. Они расположены на вкладке Data панели Toolbox. Среди них такие как:

SqlDataSource – позволяет подключаться к любому источнику данных, который имеет поставщика данных ADO.NET. К таким источникам данных относятся SQL Server, OLE DB, Oracle и другие.

ObjectDataSource – позволяет подключаться к пользовательскому классу доступа к данным. Этот подход целесообразно применять при построении крупномасштабных и сложных приложений. Подробнее об использовании пользовательских классов доступа к данным и элементе ObjectDataSource будет рассказано в разделе, посвященном реализации трехуровневой архитектуры доступа к данным в ASP.NET.

AccessDataSource – позволяет подключаться к базам данных Microsoft Access.

XmlDataSource – позволяет подключаться к XML файлу.

SiteMapDataSource – позволяет подключаться к файлу Web.Sitemap, содержащего описание навигационной структуры Веб-приложения.

9.2.2.5. Использование DataView для фильтрации и сортировки данных

При создании сложных приложений, ориентированных на активное взаимодействие с пользователями, большое значение играет отображение данных и возможности настройки этого отображения. ASP.NET предоставляет возможность такой настройки отображения извлекаемой из базы данных информации посредством использования класса DataView. Этот класс снабжен возможностью фильтрации и сортировки отображаемых данных, причем эти режимы никак не затрагивают реальные данные.

Каждый объект DataTable по умолчанию имеет ассоциированный с ним объект DataView. При этом допускается создание множества объектов DataView для

создания множества представлений одной и той же таблицы. Для обращения к представлению по умолчанию объекта DataTable необходимо обратиться к свойству DataTable.DefaultView.

Рассмотрим примеры использования основных функций DataView.

9.2.2.5.1. Сортировка данных

Сценарий использования объекта DataView выглядит следующим образом:

- создание объекта DataView;
- настройка его параметры сортировки или фильтрации;
- привязка его к элементу отображения данных;
- инициирование процесс привязки данных.

Рассмотрим пример реализации сортировки данных, извлеченных из таблицы FORTEST_Table:

```
protected void Page_Load(object sender, EventArgs e)
{
    string ConnectionString =
    WebConfigurationManager.ConnectionStrings["FORTEST"].ConnectionString;
    SqlConnection con = new SqlConnection(ConnectionString);
    string query = "SELECT * FROM FORTEST_Table";
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    DataSet ds = new DataSet();
    da.Fill(ds, "Columns");
    DataView dv = new DataView(ds.Tables["Columns"]);
    dv.Sort = "Column1";
    GridView1.DataSource = dv;
    Page.DataBind();
}
```

Как видно из примера, сортировка данных заключается просто в присваивании свойству Sort объекта DataView выражения сортировки. В данном примере, выражение сортировки состоит из имени одного столбца таблицы. В общем случае, оно может включать несколько столбцов, разделенных запятыми. По умолчанию производится сортировка в порядке возрастания значения элементов столбца. При необходимости, возможно изменение порядка сортировки.

9.2.2.5.2. Фильтрация данных

Для выполнения фильтрации данных при помощи DataView необходимо воспользоваться свойством RowFilter. Содержимое RowFilter очень напоминает команду Where запроса SQL. Здесь записывается условие фильтрации набора данных. При этом возможно использование тех же операций сравнения и логических операций, которые используются в языке SQL. Кроме того, здесь возможно выполнение простейших вычислительных операций, таких как сложение, вычитание, умножение, деление, вычисление остатка от деления.

9.2.3. Ключевые термины

Microsoft SQL Server, ADO.NET.

9.3. Краткие итоги

База данных – это совокупность связанных данных, организованных по определенным правилам, предусматривающим общие принципы описания, хранения и манипулирования, независимая от прикладных программ.

Наиболее часто используются следующие отличительные признаки БД:

- база данных хранится и обрабатывается в вычислительной системе;

- данные в базе данных логически структурированы (систематизированы);
- база данных включает метаданные, описывающие логическую структуру БД.

Система управления базами данных (СУБД) – это специализированная программа (чаще комплекс программ), предназначенная для организации и ведения базы данных.

Обычно современная СУБД содержит следующие компоненты:

- ядро;
- процессор языка базы данных;
- подсистему поддержки времени исполнения;
- сервисные программы.

Модель данных есть формальная теория представления и обработки данных в системе управления базами данных (СУБД), которая включает, по меньшей мере, три аспекта:

- структура данных;
- набор допустимых операций, выполняемых на структуре данных;
- ограничения целостности.

Классификация БД по модели данных:

- иерархические;
- сетевые;
- реляционные;
- постреляционные;
- объектно-ориентированные;
- объектно-реляционные.

Реляционная модель данных (РМД) – логическая модель данных, прикладная теория, описывающая структурный аспект, аспект целостности и аспект обработки данных в реляционных базах данных.

Основные понятия реляционных БД:

- Таблицы;
- Первичные и внешние ключи;
- Индексирование;
- Нормальные формы:
- Первая нормальная форма (1NF);
- Вторая нормальная форма (2NF);
- Третья нормальная форма (3NF);
- Нормальная форма Бойса-Кодда (BCNF);
- Транзакции.

Проектирование баз данных – процесс решения класса задач, связанных с созданием баз данных.

При обеспечении Веб-доступа к существующим БД, возможен ряд путей – комплексов технологических и организационных решений:

- преобразование содержимого БД в статические документы;
- динамическое создание гипертекстовых документов;
- создание информационного хранилища на основе СУБД.

SQL является инструментом, предназначенным для обработки и чтения данных, содержащихся в компьютерной базе данных.

Язык SQL делится на несколько частей:

- операторы определения данных;
- операторы манипуляции данными;
- операторы определения доступа к данным;
- операторы управления транзакциями;

- операторы определения курсора.

Язык SQL имеет множество достоинств и ряд недостатков.

Производители СУБД вводят различные расширения SQL.

В качестве основных технологий .NET для доступа к данным выступают:

- ADO.NET;
- Linq;
- ADO.NET Entity Framework.

Microsoft SQL Server – система управления реляционными базами данных (СУБД), разработанная корпорацией Microsoft.

ADO.NET представляет собой набор библиотек, входящих в Microsoft .NET Framework предназначенных для взаимодействия с различными хранилищами данных из .NET приложений.

ADO.NET содержит подсоединенные объекты:

- Connection;
- Transaction;
- DataAdapter;
- Command;
- Parameter;
- DataReader;

и отсоединенные:

- DataSet;
- DataTable;
- DataColumn;
- DataRow;
- DataView;
- DataRelation.

Одной из основных идей, лежащих в основе ADO.NET является наличие поставщиков данных.

Наиболее типичным сценарием работы Веб-приложения с БД является следующий:

- устанавливается соединение и открывается подключение к базе данных;
- выполняется один или несколько запросов;
- осуществляется отключение от источника данных.